12. End-End Communication at Transport Layer.

Aim:

a) Implement echo client server using TCP/UDP sockets.

b) Implement a chat Program using Socket Programming.

TCP

```
import socket
import threading
#------ SERVER------
def start_server():
    server = socket.socket(socket.AF_INET, socket.
                           SOCK_STREAM)
    server.bind(('127.0.0.1', 5000))
    server.listen(1)
    Print ("[SERVER] Listening on 127.0.0.1:5000")
    conn, addr = server.accept()
    Print (f"[SERVER] Connected by {addr}")
while True:
    data = conn.recv(1024).decode()
    if not data or data.lower() == 'quit':
        Print ("[SERVER] Connection closed by
                client.") break
    Print (f"[SERVER] Received : {data}")
    conn.send (f"server received : {data}".encod
    conn.close()
    server.close()
```

```
#.....CLIENT.....
def start_client():
    client = socket.socket(socket.AF_INET,
                           socket.SOCK_STREAM)
    client.connect(('127.0.0.1', 5000))
    print("[CLIENT connected to server]")

    while True:
        msg = input("Enter message (or 'quit' to
                    exit): ")
        client.send(msg.encode())
        if msg.lower() == 'quit':
            break
        response = client.recv(1024).decode()
        print(f"[SERVER RESPONSE] {response}")
    client.close()
#.....MAIN.....
if __name__ == "__main__":
    #Run server in a background thread
    server_thread = threading.Thread(
                    target = start_server,
    daemon = True)
    server_thread.start()

    # Run client in main thread
    start_client()
```

Output:

------ RESTART: C:/Users/Hala/Downloads/tcp.py ------

[SERVER] Listening on 127.0.0.1:5000 [CLIENT] Connected to sower.

UDP

```
import socket
import threading

#..... SERVER.....
def udp_sower():
    s = Socket.Socket(Socket.AF_INET, Socket
        Sock_DGRAM)
    s.bind(('127.0.0.1', 12000)
    Print("[SERVER] Running on 127.0.0.1:12000")
    while True:
      data, addr = s.recvfrom(1024)
      msg = data.decode()
      if msg.lower() == 'quit':
    Print(f"[SERVER) client {addr} ended the
Connection.")
    break
    Print(f"[SERVER] Received from {addr}:
          {msg}")
    s.sendto(data, addr) # Echo back
    s.close()
    Print("[SERVER] stopped.")
    # ---- CLIENT ----.
    def udp_client():
```

```
c = socket.socket(socket.AF_INET,
Socket.SOCK_DGRAM)
Server = ('127.0.0.1', 12000)
while True:
    msg = input("Enter message (or 'quit'
                to stop): ")
    c.sendto(msg.encode(), Server)
    if msg.lower() == 'quit':
        Print("[CLIENT] Closing Connection
                ....") break
    data, _ = c.recvfrom(1024)
    Print("Echo from Server:", data.decode()
c.close()
Print("[CLIENT] Stopped.")
# ---- MAIN -----
if __name__ == "__main__":
    threading.Thread(target = udp_server
daemon = True).Sta
    rt() UDP_client()
```

Output

```
Enter message (or 'quit' to stop): [SERVER] Runn
    on 127.0.0.1:12000 hi
[SERVER] Received from ('127.0.0.1', 49670): hi
```

Result:-

Thus we aquire the TCP/UDP for massage passing program for client & Server